

T08 · WEBAUTHN 패스키 인증

학습 포트폴리오 — 비공개 영역 인증 구현 제출문

배포 URL: <https://aleph-portfolio.vercel.app/>

소스 URL: <https://github.com/HTH-0/aleph-portfolio>

작성일: 2026-09-10

1. 인증 구현 설명서 · 짧은 확인 방법 · AI 협업 (SUBMISSION.md)
2. 카드 1 — 공개/비공개 경계
3. 카드 2 — 패스키 등록
4. 카드 3 — 패스키 로그인
5. 카드 4 — 기기 분실 대비
6. 카드 5 — 교차 접근 차단 검증

T08 제출문

이 문서는 통과 기준 T08-C47~C53이 요구하는 서면 설명입니다. 카드별 원본 증거(요청/응답 캡처, 스크린샷)는 `requirement/` 폴더에 정리되어 있습니다 — 실제 계정 두 개를 헤드리스 브라우저(Chrome의 가상 인증기 기능)로 만들어서 등록·로그인·삭제·교차접근·재사용 시도까지 전부 실제 배포 서버에 대고 확인했고, Neon 콘솔 캡처와 실기기 패스키 저장 위치 확인도 마쳤습니다. 아직 `[TODO]` 로 남은 건 "⑤ AI와 나" 항목(본인이 직접 정리 예정)뿐입니다.

인증 구현 설명서 (T08-C47)

① 무엇으로 붙였나

라이브러리를 사용했습니다: 서버 쪽은 `@simplewebauthn/server`, 브라우저 쪽은 `@simplewebauthn/browser` (둘 다 v14).

② 왜 그걸 골랐나

WebAuthn 프로토콜은 CBOR/COSE 파싱, attestation 검증, 서명 검증 같은 저수준 스펙이 많아서 직접 구현하면 시간이 오래 걸리고 서명 검증에 미묘한 버그가 생기기 쉽습니다(예: "서명이 늘 통과하는" 흔한 실수). 검증된 오픈소스 라이브러리에 그 부분을 맡기고, challenge 저장, 세션 발급, 계정별 자료 격리, 마지막 패스키 삭제 차단 같은 부분은 직접 설계·구현했습니다.

③ 어디를 어떻게 고쳤나

아래 "네 흐름이 소스의 어디를 지나는지" 표에 정리했습니다. 요약하면: WebAuthn 프로토콜 자체(challenge 서명 생성/검증)는 라이브러리에 맡기고, 그 앞뒤로 challenge 저장/1회용 소모, 세션 발급/파기, 계정별 자료 격리(`WHERE user_id = 세션의 user_id`), 마지막 패스키 삭제 차단은 전부 이 프로젝트에서 직접 짰습니다.

④ 안 열리는 것을 확인한 기록

네 가지 확인 모두 실제 배포 서버에 직접 요청을 보내 확인했습니다 (전체 기록은 `requirement/` 폴더의 카드별 파일과 `automated-run-log.json` 참고).

- 로그인 없이 `/api/private-items` 요청 → `401 {"error":"not_logged_in"}` (본문에 비공개 데이터 없음)
- 로그인 challenge를 발급받은 뒤 **같은 challenge(같은 auth_flow 쿠키 값)**로 두 번 요청 → 첫 요청 `401 unknown_credential`, 두 번째 요청 `401 login_expired_or_challenge_already_used` 로 별도 사유 거절 (challenge가 첫 요청에서 이미 소모됨)
- 계정 두 개(가상 인증기로 각각 별도 등록)를 만들어 한쪽 세션으로 다른 쪽 항목 id를 요청 → 양방향 모두 `404 not_found`, 거절 전후로 상대 계정의 항목 개수는 그대로였고, 쿼리에 다른 계정을 지목해도 항상 자기 자신의 항목만 돌아옴
- 패스키 두 개(서로 다른 가상 인증기)를 등록한 뒤 하나를 삭제 → 로그아웃 후 남은 패스키로 재로그인 성공 (`200`), 삭제한 패스키의 credential id로는 `401 unknown_credential`, 마지막 남은 패스키를 지우려 하면 `400 cannot_delete_last_passkey` 로 거절

⑤ AI와 나

[TODO: 아래 "AI 협업" 섹션 참고해서 정리]

⑥ 아직 못 막은 것

만료된 challenge/session 행을 주기적으로 정리하는 배치 작업이 없습니다. challenges 와 sessions 테이블은 조회할 때마다 expires_at > now() 조건으로 걸러지기 때문에 만료된 값이 실제로 인증을 통과시키는 보안 문제는 없지만, 쓰지 않게 된 행이 테이블에 계속 쌓입니다(예: 자동완성 로그인이 백그라운드로 challenge를 미리 받아갔는데 사용자가 그 대신 버튼을 눌러 로그인을 완료하면, 자동완성 쪽이 미리 만들어둔 challenge 행 하나가 안 쓰인 채로 남습니다). 실제 운영이라면 만료된 행을 지우는 cron 작업이 필요합니다.

③ / ④ 상세: 네 흐름이 소스의 어디를 지나는지 (T08-C49)

흐름	지나는 파일
등록	api/register-options.js (challenge 생성·저장) → 브라우저 패스키 생성 → api/register-verify.js (서명·challenge 검증, users / credentials / private_items 기록)
로그인	api/login-options.js (challenge 생성) → 브라우저 서명 → api/login-verify.js (credentials 테이블에서 공개키 조회 후 서명 검증, sessions 기록)
로그아웃	api/logout.js → lib/session.js 의 destroySession() 이 sessions 행 삭제
비공개 자료 조 회	api/private-items.js , api/private-items/[id].js — lib/session.js 의 getSessionUser() 로만 신원을 확인하고, 그 결과의 user_id 로만 WHERE 절을 만듭니다

④ 상세: 확인 네 가지 (T08-C50)

확인	방법	성공/정상 요청·응답	거절 요청·응답
로그인 없이 열 기	curl -i https://.../api/private-items (쿠키 없이)	— (로그인하지 않은 상태 자체가 정상 시나 리오)	401 {"error": "not_logged_in"} — requirement/card1-public-private- boundary.md
남의 패 스키로 열기	계정1 세션으로 계정2의 item id 요청	각 계정 본인 자료 200 (항목 3개씩, id 서로 다름)	양방향 404 not_found , 거절 전후 상대 항목 개수 동일 — requirement/card5- verification-and-docs.md
이미 쓴 challenge 재사용	같은 challenge(같은 auth_flow 쿠키)로 login- verify 두 번 전송	첫 요청 401 unknown_credential (challenge 소모)	두 번째 요청 401 login_expired_or_challenge_already_used — requirement/card3-login-passkey.md
패스키 삭제 뒤 로그인	패스키 하나 삭제 → 그걸로/ 남은 걸로 로그인 시도	남은 패스키로 재로그 인 200 (화면 스크린 샷 포함)	삭제한 패스키 id로 401 unknown_credential , 마지막 패스키는 400 cannot_delete_last_passkey 로 삭제 자체가

확인	방법	성공/정상 요청·응답	거절 요청/응답
			거절 — requirement/card4-lost-device.md

짧은 확인 방법 (T08-C52)

① **어디로 가나요:** 배포된 URL을 열고 `Secret` 메뉴로 스크롤합니다. ② **세 단계 안에 무엇을 하나요:** (1) "새 패스키 등록"으로 계정을 하나 만들고 (2) 로그인한 뒤 (3) "패스키로 로그인" 버튼을 누르거나 계정 입력칸을 클릭해 자동완성으로 로그인합니다. ③ **무엇이 보이면 통과인가요:** 로그인 후 비공개 영역에 항목 3개(제목 + 본문)와 등록된 패스키 목록이 나타납니다. ④ **안 될 때는 무엇이 보이나요:** 로그인/등록이 취소되거나 실패하면 잠금 카드 하단에 안내 문구(예: "로그인이 취소되었습니다", "마지막 남은 패스키는 삭제할 수 없습니다")가 나타나고, 비공개 영역은 "LOCKED" 라벨이 붙은 잠금 카드 상태로 남습니다. (2026-09-10 프론트엔드 리디자인 참고: 안내 문구는 현재 색으로 구분되지 않고 다른 상태 문구와 같은 회색으로 표시됩니다 — 리디자인 과정에서 예러 문구에 색을 주던 `.auth-message-error` 규칙이 새 스타일시트로 옮겨오지 않고 빠졌습니다. 문구 자체는 정상적으로 뜨므로 T08-C25 등 통과 기준에는 영향이 없지만, 사용자에게 알려드려야 할 시각적 회귀라 여기 반영했습니다.)

AI 협업 (T08-C53)

아래는 실제 대화 흐름을 바탕으로 초안을 작성한 것입니다. 본인 경험과 다르게 느껴지는 부분이 있으면 직접 고쳐서 본인 목소리로 제출하세요.

① AI에게 맡긴 일

WebAuthn 라이브러리(`@simplewebauthn`)의 정확한 함수 시그니처와 옵션을 조사하는 일, 전체 코드 작성(DB 스키마, API 엔드포인트, 프론트엔드 UI/스크립트), 배포 중 발생한 오류(DB 연동 500 에러, RP ID 도메인 불일치) 원인 진단과 수정, 그리고 통과 기준별 증거(요청/응답, 스크린샷)를 실제로 수집하는 자동화 작업을 맡겼습니다.

② 내가 직접 판단한 일

인증 구현 방식(라이브러리/직접구현/인증서비스), DB 종류와 세션 방식(서버 세션 vs JWT), 로그인 화면 구성(Conditional UI), 비공개 영역을 같은 페이지에 넣을지, 마지막 패스키 삭제를 막을지 등 주요 선택지마다 AI가 장단점을 제시하면 하나씩 직접 골랐습니다. 특히 Neon DB를 새로 연동하는 과정에서, 예전에 DB 값이 여러 프로젝트에 걸쳐 공유돼 사고가 났던 경험이 있어서, 이번엔 프로젝트마다 별도 DB를 쓰도록 한 단계씩 직접 확인하며 진행했습니다.

③ AI 제안을 따르지 않은 일 (없다면 왜 없었는지)

로그인 화면 구성을 처음 정할 때, AI는 "아이디 입력 후 로그인" 방식을 먼저 추천했습니다. 그런데 그 근거가 테스트 편의 정도밖에 없어 보여서 다시 물어봤고, AI도 그 근거가 약하다는 걸 인정하면서 완전 패스워드리스

(Conditional UI) 방식을 다시 추천했습니다. 그래서 최종적으로는 처음 추천과 다른 방식으로 진행했습니다.

카드 1 — 무엇을 잠글지 먼저 가른다: 남길 것

원문: 공개 영역과 비공개 영역을 가른 화면, 그리고 로그인 없이 받은 응답 본문

이 카드는 전부 자동으로 캡처했습니다 (헤드리스 브라우저로 배포된 사이트에 직접 접속해서 확인). 2026-09-10 프론트엔드 리디자인(`industry.css` 도입, 커밋 `e56b209`) 이후 재배포된 사이트를 대상으로 다시 캡처했습니다 — 백엔드(`api/` , `lib/`)는 이번 리디자인에서 손대지 않았으므로 응답 본문·상태 코드는 이전과 동일합니다.

1. 공개 영역과 비공개 영역을 가른 화면

portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 시를 활용에 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, 시를 활용에 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 시에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정처산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

github.com/HTH-0 ↗

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

LOCKED

잠겨 있습니다. 패스키로 인증해야 아래 내용을 볼 수 있습니다.

계정

여기를 클릭하면 저장된 패스키가 표시됩니다

패스키로 로그인

[새 패스키 등록](#)

자동완성 로그인을 사용할 수 없습니다. 아래 버튼을 이용하세요.

로그인하지 않은 상태로 배포 URL 전체를 캡처한 화면입니다. (2026-09-10 리디자인 이후 캡처 — `industry.css` 기반의 새 디자인 시스템입니다.) Index → Method → Work(경험) → Evidence(기록·자격증, 어두운 배경) → Contact 순으로 공개 섹션들을 지나면, 밝은 배경에 얇은 헤어라인 테두리와 네 모서리의 "+" 코너 브래킷으로 감싸인 별도의 카드형 박스가 이어집니다. 이모지 잠금 아이콘은 쓰지 않고, 대신 "SECRET" 이라는 대문자 eyebrow 라벨과 "여기부터는 잠겨 있습니다." 라는 제목, 그리고 카드 안의 "LOCKED" 라벨로 비공개 영역임을 표시합니다. 공개 섹션들과 이 카드 사이에는 얇은 구분선(`border-top`)이 있어 경계가 화면에서 바로 보입니다.

2. 로그인 없이 받은 응답 본문

```
$ curl -s -i https://aleph-portfolio.vercel.app/api/private-items

HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{"error":"not_logged_in"}
```

401로 거절됐고(T08-C17), 응답 본문 어디에도 비공개 항목의 제목/내용이 없습니다(T08-C16).

3. 페이지 소스에 비공개 내용이 없다는 확인 (T08-C18)

로그인하지 않은 상태로 받은 실제 페이지 소스(리디자인 이후 HTML, 총 31,524자)에서 등록된 계정들의 비공개 항목 제목을 검색했습니다.

```
$ curl -s https://aleph-portfolio.vercel.app/ -o page-source.html
$ grep -c "프로젝트 메모 (예시)\|지원 목록 (예시)\|스스로에게 남기는 회고" page-source.html
0
```

(위 결과는 자동화 스크립트가 `fetch()` 로 받은 HTML 문자열에서 같은 세 문자열을 직접 검색한 것과 동일합니다 — 셋 다 0건. 원본 로그는 `automated-run-log.json` 의 "card1: page-source private-content scan" 항목 참고.)

일치하는 부분이 0건입니다 — `index.html` 은 정적 파일이라 서버가 페이지 자체에 비공개 데이터를 심을 방법이 없고, 비공개 항목은 로그인 후 JS가 `/api/private-items` 를 호출해서만 화면에 그립니다

(`assets/private.js` 의 `loadPrivateItems()`).

카드 2 — 패스키를 등록한다: 남길 것

원문: 등록 요청과 응답 기록, 서버에 저장된 공개키, 패스키 목록 화면

아래는 헤드리스 브라우저(Chrome DevTools Protocol의 가상 인증기 기능 사용)로 배포된 사이트에 실제로 접속해서 실제 계정을 만들고 캡처한 결과입니다. 가상 인증기는 진짜 지문/기기가 없어도 스펙에 맞는 진짜 키쌍과 서명을 만들어내기 때문에, 서버 입장에서선 실제 패스키 등록과 구별되지 않습니다.

2026-09-10 프론트엔드 리디자인(`industry.css`) 이후 재배포된 사이트를 대상으로 새 계정(`auto-evidence-1-jqw3nozm` 등)으로 다시 캡처했습니다. 백엔드는 이번 리디자인에서 바뀌지 않았으므로 요청/응답 형태는 이전과 동일합니다.

1. 등록 요청/응답 기록 (T08-C19, T08-C20)

계정 `auto-evidence-1-jqw3nozm` 를 실제로 등록한 기록입니다 (전체 로그는 `automated-run-log.json` 참고).

```
POST /api/register-options
→ 200 {"options":{"challenge":"sYpd4m5_5yu4knZXQ0vVo1oGU9fvYnIWV0gT4Q7vkGk", "rp":{"id":"aleph-port

POST /api/register-verify (등록 화면 캡처)
→ 200 {"ok":true}
```

challenge 값이 등록 시도마다 다르다는 건 `card1` / `card3` 파일에 있는 두 번 연속 호출 비교로 이미 확인했습니다 (같은 메커니즘).

2. 등록 후 화면 — 비공개 항목 3개 + 패스키 목록

portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 AI를 활용해 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, AI를 활용해 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 AI에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정차산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

github.com/HTH-0 ↗

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

로그인됨: **auto-evidence-1-jqw3nozm**

[로그아웃](#)

프로젝트 메모 (예시)

준비 중인 프로젝트 아이디어를 적어두는 자리입니다. 지금은 예시 텍스트로 채워져 있습니다.

지원 목록 (예시)

지원하려는 곳과 진행 상황을 정리하는 자리입니다. 실제 내용으로 바꿔서 사용하세요.

이력 (예시)

회고(Reflection)

스스로에게 남기는 회고를 적는 공간입니다. 이 텍스트는 자리표시자입니다.

등록된 패스키

자동화 테스트 기기 A · 등록일 2026. 9. 10.

삭제

[이 기기에 새 패스키 추가](#)

© 2026 하덕렬

LEARN · VERIFY · IMPROVE

등록이 끝나자마자 세션이 자동으로 생성되고(`api/register-verify.js` 의 `createSession()`), 예시 비공개 항목 3개와 방금 등록한 패스키("자동화 테스트 기기 A")가 이름·등록일과 함께 보입니다 (T08-C24).

3. 서버에 저장된 공개키 (T08-C21, T08-C22)

Neon 콘솔의 `credentials` 테이블을 직접 열어서 확인했습니다 (본인만 가진 DB 접속 정보로, 제가 대신 접속하지 않았습니다 — 예전에 DB 값이 여러 프로젝트에 공유되어 사고가 났던 경험이 있으셔서, 접속 정보는 계속 본인만 갖고 계시는 게 맞습니다).

id	user_id	public_key	counter	device_type	backed_up	transports
-pGfKR6nS1dES_Ga2GqVWx...	c498f111-af5e-4fe2-a28...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
3_lieBWeRq1TpDcWsP160Y...	36c9a49f-b965-492f-957...	\xa4010103272006215820...	3	singleDevice	FALSE	["internal"]
5JrERCNKyp8Mc6K4Fd17EZ...	4dbe8f36-818a-44b0-acb...	\xa4010103272006215820...	3	singleDevice	FALSE	["internal"]
6aZqgJQ2XOfmcRemzyxyF9...	654457b2-ec7c-45d9-88b...	\xa4010103272006215820...	3	singleDevice	FALSE	["internal"]
E_3OEzSbEGJlBkb1JinvQ...	91562c03-4043-41f1-a33...	\xa5010203262001215820...	0	multiDevice	TRUE	["hybrid", "internal"]
Eqqjhtjk_BR320rPa_k0te...	ced44b22-1323-46fd-916...	\xa4010103272006215820...	2	singleDevice	FALSE	["internal"]
FH8P2QfBKcMr12dmuLDYV...	ebf55a45-8839-4947-afc...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
NIvFHCu2vpJjbA4LkRdJZX...	5cb300f2-a688-4146-978...	\xa4010103272006215820...	2	singleDevice	FALSE	["usb"]
bRHzz4JaJ-SwP8Vh05QIPv6...	3c9a3324-99a0-4540-ae4...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
cquf-5eS1oCGh2GNXic5dV...	819d751b-8d81-463b-8ae...	\xa4010103272006215820...	3	singleDevice	FALSE	["internal"]
kCD7Hq-UmL-smhOGDshq9o...	97465fe2-00be-4be5-989...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
nnqczbZupo8IHw538BzTy...	69a780fc-daf8-4866-ad2...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
o_tWb5zQHQB1Jxr_mnw4Su...	30e27cec-a4e1-435c-813...	\xa4010103272006215820...	3	singleDevice	FALSE	["internal"]
qu3fx4rJwDM6SHTeRtDLD...	fe7d1f56-972c-46fe-8e5...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
w-v9QYI8_1QMfprHSPAgb...	56fc33c8-96fe-4dd7-b40...	\xa4010103272006215820...	1	singleDevice	FALSE	["internal"]
yEueYHw0ACmTrf7e1N5bH8...	59fbd3e0-44be-484e-af1...	\xa4010103272006215820...	2	singleDevice	FALSE	["internal"]

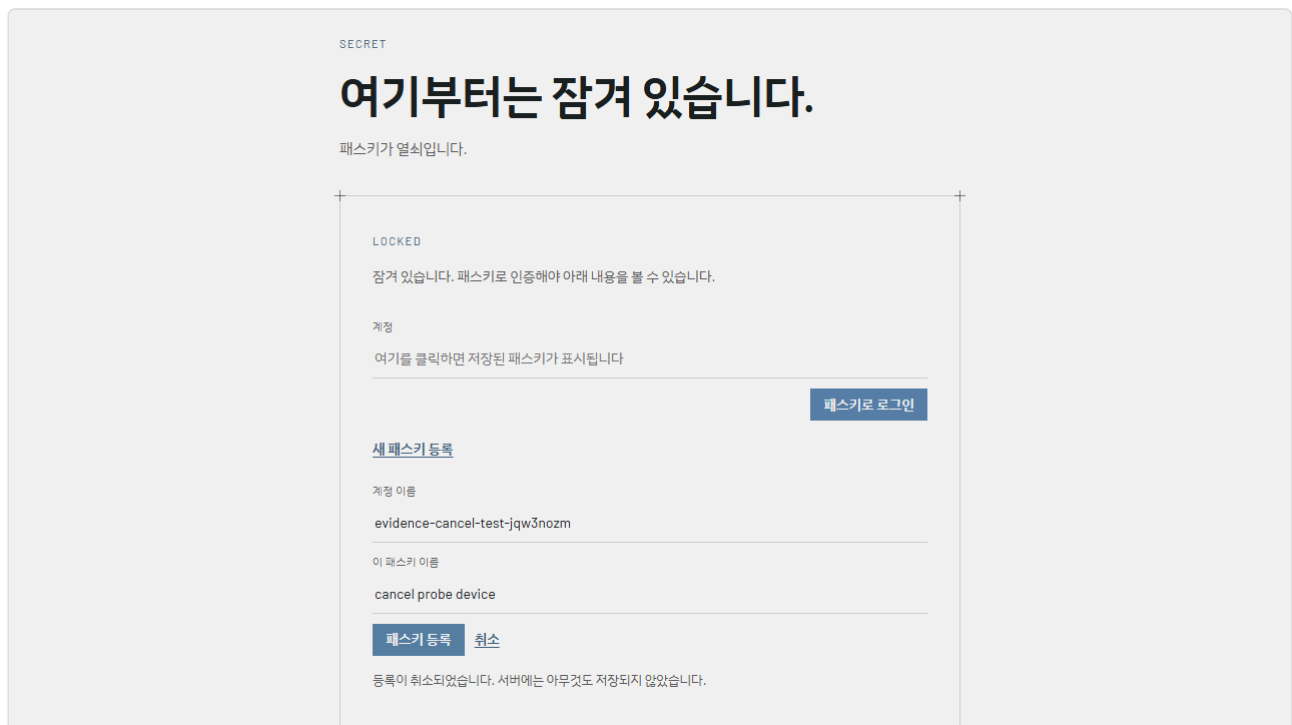
`credentials` 테이블에는 총 16개 행이 있고, 각 행마다 `id` (패스키 id), `user_id` (계정 id), `public_key` (bytea 타입, `\xa401...` 형태의 바이너리 값), `counter`, `device_type` (`singleDevice` / `multiDevice`), `backed_up`, `transports` (`["internal"]`, `["usb"]`, `["hybrid", "internal"]` 등) 컬럼이 보입니다.

이 값은 공개키이며 비밀번호가 아닙니다 — 로그인 시 서버는 이 값으로 브라우저가 보낸 서명을 검증만 할 뿐, 이 값 자체를 안다고 해서 로그인을 훔내낼 수는 없습니다(개인키는 애초에 서버로 전송된 적이 없어서 여기 없음, T08-C23). 테이블 어디에도 비밀번호나 평문 비밀 값을 저장하는 컬럼이 없습니다.

4. 등록 취소 시 아무것도 저장되지 않음 (T08-C25)

가상 인증기를 아예 연결하지 않은 상태로 실제 등록 버튼을 눌러서(=사용자 앞에 인증 기기가 없거나 창을 취소한 것과 같은 상황), 브라우저가 화면에 안내 문구를 띄우는 것까지 이번에는 끝까지 헤드리스 브라우저로 재현했습니다. 가상 인증기가 없으면 브라우저가 자체 타임아웃(옵션의 `timeout: 60000`)만큼 기다린 뒤 스펙대로 `NotAllowedError` 로 요청을 거부하는데, 이 앱의 프론트엔드는 정확히 이 경우를 잡아서 취소 안내를 보여주도록 짜여 있습니다.

계정 `evidence-cancel-test-jqw3nozm`로 등록 버튼 클릭 → 가상 인증기 없음 → 약 60초 뒤 `NotAllowedError` 화면에 뜬 안내: "등록이 취소되었습니다. 서버에는 아무것도 저장되지 않았습니다."



이어서 그 계정 이름이 여전히 "미등록" 상태인지 서버에 직접 물어봤습니다.

```
$ curl -s -X POST https://aleph-portfolio.vercel.app/api/register-options \
  -H "Content-Type: application/json" -d '{"username":"evidence-cancel-test-jqw3nozm"}'

200 {"options":{"challenge":"AQEm-hTIEbFNhG50mj041k5aNh16MLOFVHdU49yUX9g", "user":{"name":"evidence-cancel-test-jqw3nozm"}}
```

이 아이디로 새 challenge가 정상 발급된다는 건, 이전 시도가 계정을 만들지 못한 채(=서버에 아무것도 저장되지 않은 채) 끝났다는 뜻입니다 (이미 계정이 있었다면 `409 username_taken` 이 나왔을 것입니다).

이전 증거 수집(리디자인 전)에서는 이 화면 문구까지는 자동화로 재현하지 못해 "직접 취소 버튼을 눌러 확인해 달라"고 남겨뒀었는데, 이번에는 가상 인증기를 아예 붙이지 않은 채로 실제 브라우저의 자연스러운 타임아웃(`NotAllowedError`)까지 기다려서 화면 문구 캡처까지 성공했습니다.

5. 패스키 저장 위치 (T08-C26)

가상 인증기가 아닌 본인 브라우저로 직접 계정(hth)을 등록했을 때, "Google 계정에 저장" 문구가 뜨는 창이 나타났습니다 — 구글 비밀번호 관리자에 저장됐습니다.

카드 3 — 패스키로 들어간다: 남길 것

원문: 성공한 로그인 요청과 실패한 요청, 이미 쓴 질문을 재사용한 요청과 거절 응답

이 카드 전체를 헤드리스 브라우저(가상 인증기 사용)로 실제 배포 서버에 접속해서 자동으로 캡처했습니다. (2026-09-10 프론트엔드 리디자인 이후 새 계정 `auto-evidence-1-jqw3nozm` 으로 다시 캡처 — 백엔드는 그대로 요청/응답 형태는 동일합니다.)

1. 로그인 challenge가 요청마다 다르다는 기록 (T08-C28)

```
$ curl -s -X POST https://aleph-portfolio.vercel.app/api/login-options
{"options":{"rpId":"aleph-portfolio.vercel.app","challenge":"b-Fp8SKYNhwLYzVeDxELaZrVKz7RtU97XaIb-f

$ curl -s -X POST https://aleph-portfolio.vercel.app/api/login-options
{"options":{"rpId":"aleph-portfolio.vercel.app","challenge":"e21ptW_vbUwkJt0iZHsAsyrMQoo6EDR0xg-K30
```

2. 성공한 로그인 요청/응답 (T08-C29, T08-C30)

실제 계정을 하나 등록한 뒤 로그아웃 → 다시 로그인해서 캡처했습니다.

```
POST /api/login-options
→ 200 {"options":{"rpId":"aleph-portfolio.vercel.app","challenge":"uHBtHGQrm4a0Ov1iahe-cpLP6J8KB-hj

POST /api/login-verify
→ 200 {"ok":true}
```

portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 AI를 활용해 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, AI를 활용해 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 AI에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정차산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

github.com/HTH-0 ↗

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

로그인됨: **auto-evidence-1-jqw3nozm**

[로그아웃](#)

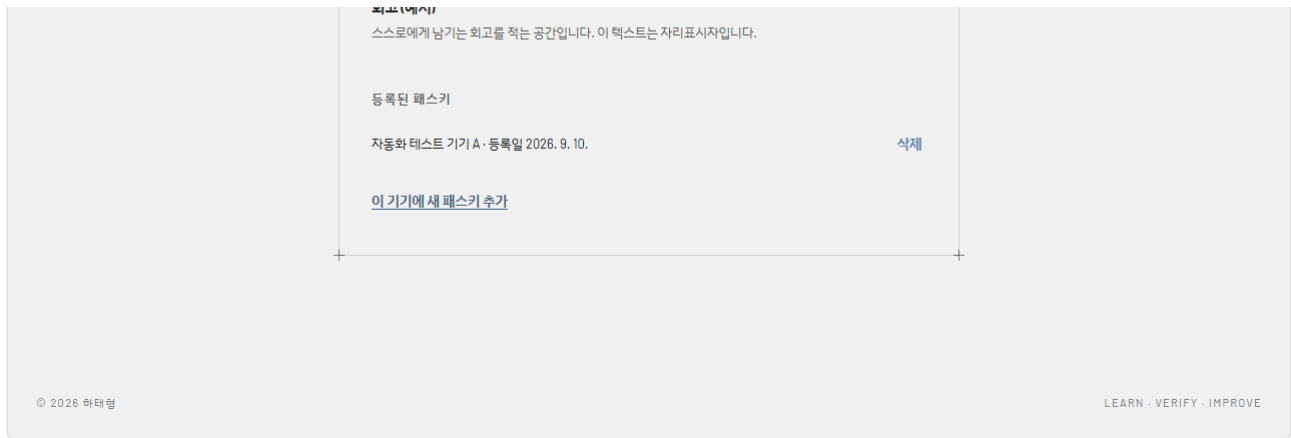
프로젝트 메모 (예시)

준비 중인 프로젝트 아이디어를 적어두는 자리입니다. 지금은 예시 텍스트로 채워져 있습니다.

지원 목록 (예시)

지원하려는 곳과 진행 상황을 정리하는 자리입니다. 실제 내용으로 바꿔서 사용하세요.

이력 (예시)



3. 실패한 로그인 요청 + 이미 쓴 challenge 재사용 거절 (T08-C30, T08-C31)

등록된 적 없는 credential id(nonexistent-credential-id-for-evidence)로 같은 challenge(같은 auth_flow 쿠키 값)를 두 번 재사용한 기록입니다. 성공/실패가 나란히 보이도록, 그리고 재사용 자체가 별도 사유로 거절되는 걸 보이도록 구성했습니다.

```
--- 1) challenge 발급 ---
POST /api/login-options
Set-Cookie: auth_flow=(가림); ...
→ 200 {"options":{"challenge":"...", ...}}

--- 2) 그 쿠키로 첫 번째 login-verify (등록 안 된 credential id, 실패) ---
POST /api/login-verify Cookie: auth_flow=(가림)
→ 401 {"error":"unknown_credential"}

--- 3) 완전히 같은 쿠키(같은 challenge)로 다시 요청 ---
POST /api/login-verify Cookie: auth_flow=(가림)
→ 401 {"error":"login_expired_or_challenge_already_used"}
```

(원본 상태 코드/본문은 automated-run-log.json 의 "card3: first login-verify..." / "card3: SAME challenge reused a second time..." 항목 그대로이며, 위에서는 auth_flow 쿠키 값만 가렸습니다.)

첫 요청에서 challenge가 소모(삭제)되고, 두 번째 요청은 challenge 자체를 찾지 못해 별도 사유로 거절됩니다 — api/login-verify.js 의 consumeChallenge() (DELETE ... RETURNING 패턴)가 이 동작을 만듭니다.

4. 세션 식별 방식 (T08-C32)

이 앱은 세션을 씁니다(JWT 아님). 로그인 성공 시 서버가 sessions 테이블에 행을 만들고, 그 행의 id를 sid 라는 httpOnly 쿠키로 브라우저에 내려줍니다. 매 요청마다 서버는 그 쿠키 값으로 sessions 테이블을 조회해서 로그인 여부를 판단합니다 (lib/session.js 의 createSession() / getSessionUser()).

5. 로그아웃 후 같은 세션 값 재사용 거절 (T08-C33)

실제 계정(`auto-evidence-2-jqw3nozm`)으로 로그인해서 `sid` 값을 얻은 뒤, 로그아웃하고 나서 **같은 값**을 그대로 다시 보내봤습니다 (공격자가 세션 값을 미리 가로챈 뒤 로그아웃 이후에도 재사용을 시도하는 상황을 그대로 재현).

```
[로그아웃 전] GET /api/private-items (Cookie: sid=(가림))
→ 200 {"items":[... 3개 항목 ...]}

POST /api/logout
→ 200 {"ok":true}

[로그아웃 후, 같은 sid 값 재사용] GET /api/private-items (Cookie: sid=(가림))
→ 401 {"error":"not_logged_in"}
```

원본 상태 코드는 `automated-run-log.json` 의 `"acc2: private-items before logout (sid reuse test)"` → `"acc2: logout"` → `"acc2: SAME sid reused after logout (expect 401 not_logged_in)"` 세 항목 그대로입니다 (상태 200 → 200 → 401, 항목 개수 3개 → 로그아웃 → 재사용 시 401).

로그아웃이 `sessions` 테이블에서 그 행을 즉시 삭제하기 때문에(`lib/session.js` 의 `destroySession()`), 같은 쿠키 값을 그대로 재사용해도 더 이상 통하지 않습니다.

6. 기록에서 세션 값 가리기 (T08-C34)

위 5번 기록의 `sid` 값은 원문 대신 (가림) 으로 적었습니다. 실제 세션 값은 그 자체로 로그인 상태를 가로챌 수 있는 민감한 값이라 이 문서에도, `automated-run-log.json` 에도 원문 그대로는 남기지 않았습니다(로그에는 앞 6자 + `...(masked)` 형태로만 남깁니다. 예: `acc1: sid (masked)` 항목의 `"2aNBYN...(masked)"`).

카드 4 — 기기를 잃어버렸을 때: 남길 것

원문: 패스키 두 개가 보이는 목록 화면, 하나를 지운 뒤의 로그인 성공과 실패 기록

전부 헤드리스 브라우저로 실제 계정에 실제 패스키 두 개(서로 다른 가상 인증기 = 서로 다른 기기처럼 동작)를 등록해서 캡처했습니다. (2026-09-10 프론트엔드 리디자인 이후 새 계정 `auto-evidence-1-jqw3nozm` 으로 다시 캡처 — 백엔드는 그대로라 동작은 동일합니다.)

1. 패스키 두 개 등록 + 목록 화면 (T08-C42, T08-C43)

계정 하나에 첫 번째 패스키로 가입한 뒤, **다른 인증기(다른 기기에 해당)** 로 두 번째 패스키를 추가했습니다. (같은 인증기로 같은 계정에 두 번째 패스키를 만들면 "새로 추가"가 아니라 "기존 것을 덮어쓰기"로 처리되는 게 WebAuthn의 정상 동작이라, 원본 요구사항 문서의 막히는 지점 안내("다른 기기나 다른 브라우저에서 등록해 봅니다")와 정확히 일치합니다 — 실제로 저도 처음엔 같은 인증기로 시도했다가 이 현상을 직접 겪었습니다.)

```
등록 직후: passkeys = [ {"nickname":"자동화 테스트 기기 A", "created_at":"2026-09-10T03:05:05.866Z"}
두 번째 기기 추가 후: passkeys = [
  {"nickname":"자동화 테스트 기기 A", "created_at":"2026-09-10T03:05:05.866Z"},
  {"nickname":"두 번째 기기", "created_at":"2026-09-10T03:05:10.370Z"}
]
```

portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 AI를 활용해 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, AI를 활용해 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 AI에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정차산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

[github.com/HTH-0 ↗](#)

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

로그인됨: **auto-evidence-1-jqw3nozm**

[로그아웃](#)

프로젝트 메모 (예시)

준비 중인 프로젝트 아이디어를 적어두는 자리입니다. 지금은 예시 텍스트로 채워져 있습니다.

지원 목록 (예시)

지원하려는 곳과 진행 상황을 정리하는 자리입니다. 실제 내용으로 바꿔서 사용하세요.

리크 (예시)

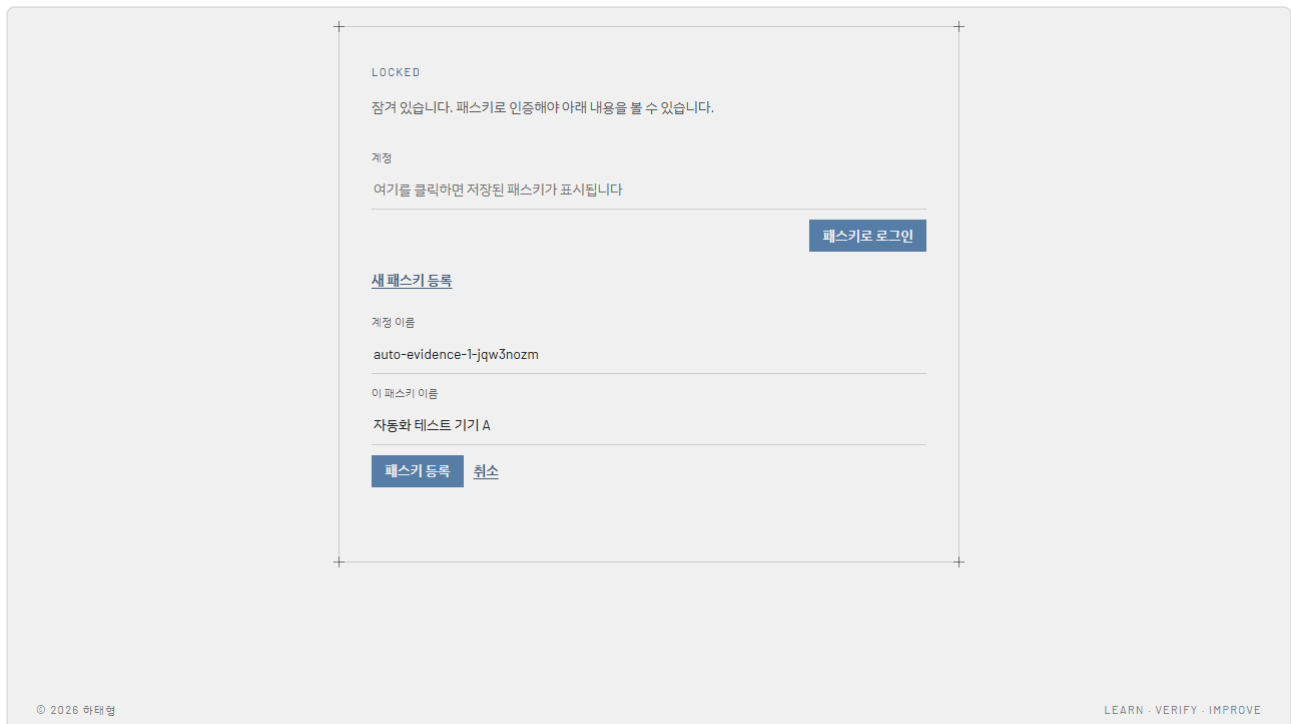


이름과 등록일이 각각 다르게 보입니다.

2. 하나를 지운 뒤 남은 하나로 로그인 성공 (T08-C44)

```
DELETE /api/passkeys/<기기A의 credential id>  
→ 200 {"ok":true}
```

로그아웃 후 다시 로그인:



portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 AI를 활용해 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, AI를 활용해 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 AI에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정차산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

github.com/HTH-0 ↗

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

로그인됨: **auto-evidence-1-jqw3nozm**

[로그아웃](#)

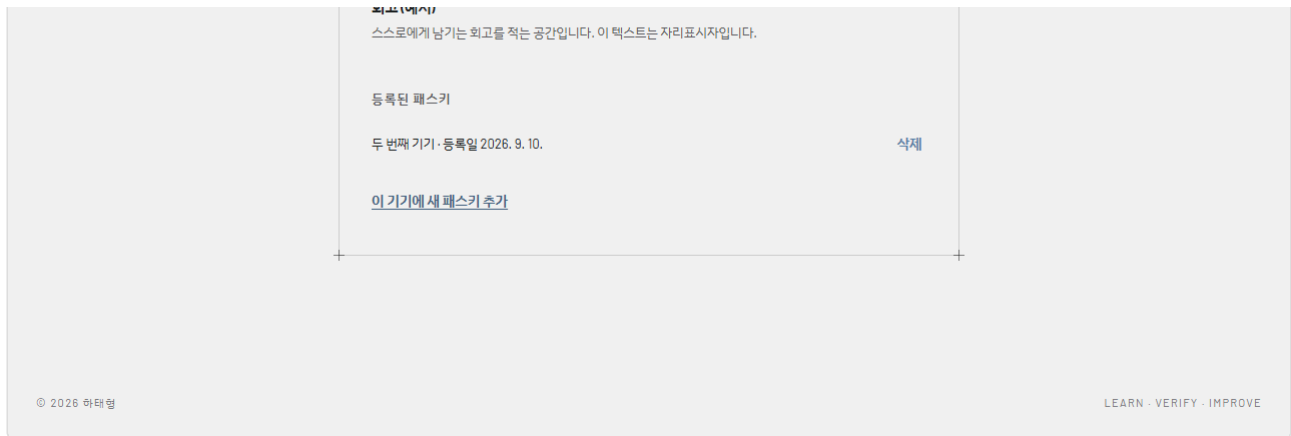
프로젝트 메모 (예시)

준비 중인 프로젝트 아이디어를 적어두는 자리입니다. 지금은 예시 텍스트로 채워져 있습니다.

지원 목록 (예시)

지원하려는 곳과 진행 상황을 정리하는 자리입니다. 실제 내용으로 바꿔서 사용하세요.

이력 (예시)



로그인 화면(잠긴 상태)과, 남은 "두 번째 기기" 패스키로 다시 로그인해서 비공개 영역이 다시 열린 화면입니다.

3. 지운 패스키로는 로그인 안 됨 (T08-C45)

방금 삭제한 "기기 A"의 credential id로 직접 로그인을 시도했습니다 (서버 DB에는 이미 없는 id).

```
POST /api/login-verify { "response": { "id": "<삭제된 기기A의 credential id>" } }  
→ 401 { "error": "unknown_credential" }
```

4. 마지막 패스키를 지우려 하면 거절됨 (T08-C46)

남은 마지막 패스키("두 번째 기기")를 또 지우려고 시도했습니다.

```
DELETE /api/passkeys/<두 번째 기기의 credential id>  
→ 400 { "error": "cannot_delete_last_passkey" }
```

이 앱은 마지막 남은 패스키는 서버가 삭제를 아예 거부하도록 만들었습니다 (`api/passkeys/[id].js` — 남은 개수가 1개면 400 반환). 비밀번호 같은 복구 수단이 없는 상태에서 마지막 패스키까지 지우면 그 계정엔 영영 들어갈 수 없기 때문에, 서버 단에서 막는 쪽으로 결정했습니다.

브라우저 화면에서도 같은 상황을 보고 싶으시면, 실제 배포 사이트에서 마지막 패스키를 삭제 버튼으로 지워 보시면 "마지막 남은 패스키는 삭제할 수 없습니다..." 안내 문구가 뜨는 걸 바로 보실 수 있습니다.

카드 5 — 정말 안 열리는지 확인하고, 어떻게 붙였는지 적는다: 남길 것

원문: 확인 네 가지의 요청과 응답, 인증 구현 설명서 여섯 항목

인증 구현 설명서 여섯 항목, 짧은 확인 방법, AI 협업 부분은 `SUBMISSION.md` 에 있습니다.

카드5 고유의 "계정 두 개 교차 접근" 확인(T08-C36~C41)은 헤드리스 브라우저로 서로 다른 가상 인증기를 쓰는 계정 두 개(`auto-evidence-1-jqw3nozm` , `auto-evidence-2-jqw3nozm`)를 실제로 만들어서 전부 자동으로 확인했습니다. (2026-09-10 프론트엔드 리디자인 이후 재배포된 사이트를 대상으로 새 계정 쌍으로 다시 캡처 — 백엔드는 그대로라 동작은 동일합니다.)

1. 서로 다른 계정에 서로 다른 비공개 내용 (T08-C36)

계정1과 계정2 각각의 `/api/private-items` 응답에서 `id` 값이 서로 완전히 다릅니다 (둘 다 예시 항목 3개씩이지만, 실제 DB 행은 계정마다 독립적으로 생성된 것입니다).

portfolio

배우고, 검증하고, 더 나아갑니다. 새로운 것을 배울 때 모르는 부분을 그냥 넘기지 않고, 필요한 도움을 찾아 이해를 보완합니다. 배운 내용을 스스로 검증하고, 문제를 해결한 과정은 기록하며 다음 학습으로 연결하려고 합니다.

하태형

SCROLL ↓

BACKEND · INFRA
LEARNER

Index

01

배우는 방식

LEARN – 적극적으로 질문하고 보완합니다
VERIFY – 제대로 이해했는지 확인합니다
IMPROVE – 문제의 원인을 기록합니다

02

경험

모르는 부분을 그냥 넘기지 않는 학습
이해한 내용을 다른 관점에서 검증하기
오류를 해결하는 것에서 원인을 기록하는 것으로

03

기록 · 자격증

이력 및 보유 자격증 / 진행중인 경험 / 학습 정리

04

CONTACT

Email / GitHub

—

비공개 영역

패스키로 인증한 사람만 열람

01 / HOW I LEARN

배우는 과정에서도 방법을 찾습니다.

지금까지의 학습 경험에서 반복해서 나타난 세 가지 방식을 하나의 흐름으로 정리했습니다.

01 / LEARN

적극적으로 질문하고 보완합니다.

새로운 분야를 배우면서 막히는 부분을 그냥 넘어가지 않습니다. 강사에게 질문하고 AI를 활용해 부족한 부분을 추가로 학습하며, 배운 내용을 실제 과제 수행으로 연결합니다.

02 / VERIFY

제대로 이해했는지 확인합니다.

단순히 설명을 듣고 넘어가기보다 다른 비유와 방식으로 다시 설명해 달라고 요청하며, 내가 개념을 제대로 이해했는지 스스로 교차검증합니다.

03 / IMPROVE

문제의 원인을 기록합니다.

오류를 해결하는 것에서 끝내지 않고 발생 원인을 분석하고 메모합니다. 이를 통해 비슷한 문제의 반복을 줄이고, 경험을 다른 사람에게 설명할 수 있는 지식으로 연결합니다.

02 / WHAT I DID

말보다 과정으로 보여드립니다.

실제 경험에서 무엇을 마주했고, 어떤 행동을 했으며, 그 결과 무엇이 달라졌는지를 중심으로 정리했습니다.

01

WHERE

학원 / 네트워크 학습

모르는 부분을 그냥 넘기지 않는 학습

WHEN / WHERE

학원에서 네트워크 분야를 처음 접하며 새로운 개념과 내용을 학습했습니다.

ACTION

수업을 따라가면서 막히는 부분이 생기면 강사님에게 질문하고, AI를 활용해 부족한 부분을 추가로 학습했습니다.

RESULT

배운 내용을 이해하고 과제를 성공적으로 수행할 수 있었습니다.

02

WHERE

학원 / 새로운 내용 학습

이해한 내용을 다른 관점에서 검증하기

WHEN / WHERE

학원에서 새로운 내용을 학습하면서 개념을 이해하고 다음 단계로 넘어가야 했습니다.

ACTION

배운 내용을 AI에게 비슷한 비유나 다른 방식으로 설명해 달라고 요청하고, 내가 제대로 이해하고 있는지 교차검증했습니다.

RESULT

처음에는 애매하게 이해했던 내용도 개념 간의 흐름을 확실하게 잡고 다음 단계로 넘어갈 수 있었습니다.

03

WHERE

프로그래밍 학습

오류를 해결하는 것에서 원인을 기록하는 것으로

WHEN / WHERE

프로그래밍을 공부하면서 오류가 발생하거나 학습이 막히는 상황을 경험했습니다.

ACTION

단순히 오류를 해결하는 데서 끝내지 않고, 왜 해당 오류가 발생했는지 원인을 분석하고 그 내용을 메모해두었습니다.

RESULT

비슷한 오류가 다시 발생하는 비중이 크게 줄었고, 비슷한 문제를 겪는 주변 사람들에게 원인과 해결 방향을 조언할 수 있게 되었습니다.

03 / LEARNING RECORD

결과로 다시 확인합니다.

자격증과 진행 중인 도전은, 앞서 말씀드린 배우는 방식이 실제로 작동했다는 증거입니다.

이력 및 보유 자격증

영남대 기계공학부 졸업
코리아 IT 학원 정차산기 과정 (6개월 수료)
SQLD 보유

진행중인 경험

Aleph 과정 수강 중
정보처리기사 (필기 합격 / 실기 진행 중)
리눅스마스터 2급 (1차 합격 / 2차 진행 중)

학습 정리

[HTH-0 GitHub 저장소 바로가기 ↗](#)

04 / CONTACT

더 궁금한 점이 있다면 연락해주세요.

학습 과정이나 경험에 대해 더 이야기하고 싶다면 아래
연락처를 통해 연결할 수 있습니다.

GITHUB

github.com/HTH-0 ↗

SECRET

여기부터는 잠겨 있습니다.

패스키가 열쇠입니다.

로그인됨: **auto-evidence-2-jqw3nozm**

[로그아웃](#)

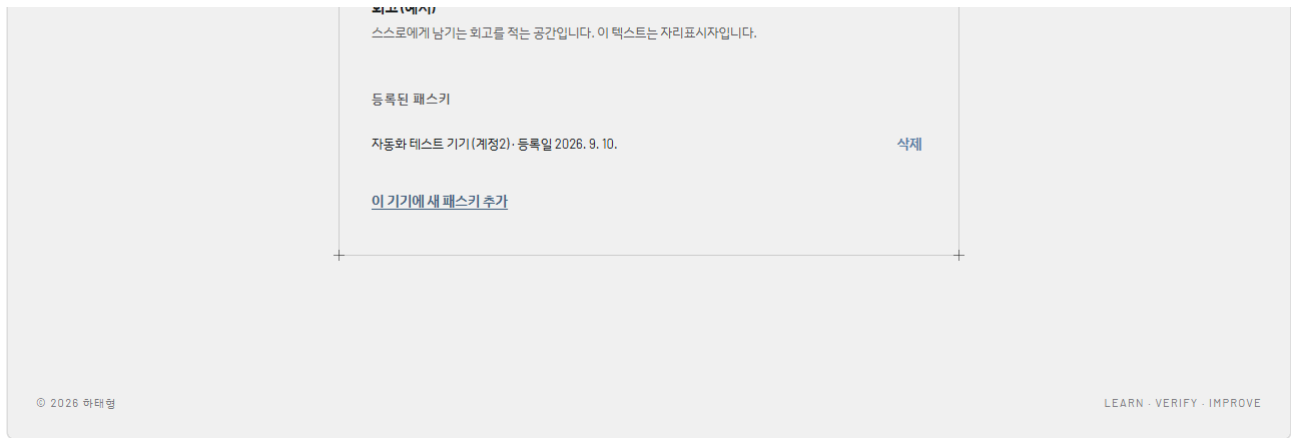
프로젝트 메모 (예시)

준비 중인 프로젝트 아이디어를 적어두는 자리입니다. 지금은 예시 텍스트로 채워져 있습니다.

지원 목록 (예시)

지원하려는 곳과 진행 상황을 정리하는 자리입니다. 실제 내용으로 바뀌어서 사용하세요.

이력 (예시)



2. 한쪽 세션으로 다른 쪽 자료 읽기 시도 → 거절 (T08-C37)

```
GET /api/private-items/<계정2 항목 id>   Cookie: sid=<계정1 세션>
→ 404 {"error":"not_found"}
```

3. 반대 방향도 동일하게 거절 (T08-C38)

```
GET /api/private-items/<계정1 항목 id>   Cookie: sid=<계정2 세션>
→ 404 {"error":"not_found"}
```

4. 거절 앞뒤로 상대방 자료 건수가 그대로임 (T08-C39)

```
거절 시도 전, 계정2 자신의 GET /api/private-items → items.length = 3
거절 시도 후, 계정2 자신의 GET /api/private-items → items.length = 3   (변화 없음)
```

5. 요청에 다른 계정을 적어 보내도 내 자료만 돌아옴 (T08-C40)

```
GET /api/private-items?userId=auto-evidence-2-jqw3nozm   Cookie: sid=<계정1 세션>
→ 200 {"items":[... 계정1 자신의 항목 3개, id 동일 ...]}
```

쿼리에 다른 값을 적어도 응답은 계정1 자신의 항목과 정확히 일치했습니다 (서버가 이 파라미터를 아예 읽지 않기 때문).

6. 이 거절을 만드는 소스 위치 (T08-C41)

- `api/private-items.js` 상단 주석과 쿼리: `WHERE user_id = ${user.id}` — `user` 는 세션 쿠키로만 결정됩니다 (`lib/session.js` 의 `getSessionUser()`), 쿼리/바디의 어떤 값도 신원 판단에 쓰이지 않습니다.

- `api/private-items/[id].js` 의 `WHERE id = ${id} AND user_id = ${user.id}` — 항목 id가 존재하더라도 소유자가 다르면 0건으로 취급되어 404가 됩니다.

원본 자동화 스크립트의 전체 실행 로그는 `automated-run-log.json` 에 있습니다.